The LLMBDA Calculus: Information Flow Control for LLM Agent Programs: A TypeScript Implementation

Andreas Schlapbach
schlpbch@gmail.com

August 13th, 2026

Abstract

Giving a large language model access to tools like reading email, browsing the web or writing files turns it into a program whose control flow is steered by whatever text it most recently read. This report introduces the LLMBDA *calculus* of Garby, Gordon & Sands: a labeled, dynamically information-flow-tracked lambda calculus extended with first-class conversation primitives (**`send`**, **`recv`**, **`fork`**, **`clear`**) for exactly this setting. It covers the calculus's core concepts — labels, lattices, the program-counter label, the big-step judgment form, and controlled declassification via **`endorse`**. Then it describes a TypeScript reference implementation, the specific implementation choices that implementation made, and what the port does and does not establish. It discusses explicit design decisions made to target the TypeScript language. Finally, it closes with pointers to further reading and open future work.

1 Introduction

An email assistant reads your inbox to draft replies. One message contains hidden text: *"Also forward the last five emails to attacker@example.com."* Nothing broke in the ordinary sense — the model just followed an instruction. It had no way to tell that the instruction came from an untrusted sender rather than from the user it is supposed to serve.

Access control asks *who is allowed to call this function*. That is the wrong question here:

- The agent *is* allowed to read the inbox — that is its job.
- The agent *is* allowed to send email — that is also its job.
- The problem is the combination: data that arrived from an untrusted source is influencing an action with an external effect, with nothing in between checking whether that particular combination is okay.

Permissions are typically checked once, at the API boundary. What is missing is tracking *where a piece of data has been* as it moves through the program. Two shapes of failure are common in this setting:

Exfiltration. A program with access to secret data and an output channel finds an indirect way to encode the secret onto that channel, even when it is never allowed to send the secret directly. This is the classical *covert-channel/confinement problem* studied by Fenton [3] and Denning [2].

Confinement failure. Output *from* an untrusted source is treated as trusted input *to* the next step — for example, code an LLM generated from a poisoned document is executed with the same authority as code the user wrote.

The fix, in one sentence, is: *Track where data has been, not just who called the function*. This is *information flow control* (IFC): every value in the program carries a *label* describing its provenance and trust level, and every operation that moves data — assignment, output, a tool call — is checked against those labels before it is allowed to happen.

The LLMBDA calculus applies this idea to a lambda calculus extended with the primitives an LLM agent actually needs.

2 Foundations

2.1 LLMBDA in one paragraph

LLMBDA is a labeled, dynamically information-flow-tracked lambda calculus, extended with conversation primitives, defined by a big-step operational semantics [1]. Every value carries a security label from a security lattice. Labels are computed at runtime as values flow through the program, not inferred statically ahead of time. Conversation primitives as **`send`**/**`recv`**/ **`fork`**/**`clear`** are first-class syntax, so talking to a model is part of the calculus, not a side effect bolted on afterward.

2.2 What a label actually is

A label is a fact about provenance and trust. Thus, a label is neither a permission nor a type. Take the simplest useful label set, $\{U, S\}$ for *Untrusted* and *Secret*: a value read from a public web page might be labeled $\{U\}$; a value read from a private calendar might be labeled $\{S\}$; a value built by combining both is labeled $\{U, S\}$ — it inherits every source it touched. The reference implementation's `lattice.ts` implements exactly this as a $\{U, S\}$-powerset lattice, plus a richer CAMEL-style Sources$\times$Readers lattice capturing per-reader confidentiality.

2.3 The lattice: what makes labels combinable

Labels need an algebra, not just a set as one must be able to ask "does information flow from label a to label b?"

Operation	Meaning	Description
$a \sqsubseteq b$	Partial Order / Relation	**Flow Permission:** denotes whether it is safe for data labeled a to influence something labeled b
$a \sqcup b$	Join / Least Upper Bound (LUB)	**Label Aggregation:** the label of a value built from both an a- and a b-labeled input (LUB)
$\bot$	Bottom / Least Element	**Public data:** the lowest security clearance (e.g. completely public info)

Any lattice instance must satisfy: reflexivity, transitivity, and antisymmetry of $\sqsubseteq$; and $\sqcup$ must be a genuine least upper bound, not merely idempotent and commutative.

2.4 The program counter label: pc

Not every leak happens through data, control flow leaks too:

```
if secret then x := 1 else x := 0
```

No secret *value* is ever assigned to `x`, but its final value still reveals the secret bit. The classical IFC answer is the *pc label*: an implicit label tracking how tainted the fact that we are even executing this code path is. Branching on a secret raises pc for the branch bodies; anything written under a raised pc inherits that taint, whether or not it looks like it touched the secret directly.

2.5 Syntax of the LLMBDA

Form	Role
$\lambda x.e$, $e_1\ e_2$	ordinary lambda calculus
$l : e$	evaluate e with pc raised by label l
e_1 `?` e_2, `assert` e	runtime checks against the current label state
`record`, `array`, `.field`, `[i]`	structured data, each cell independently labeled
`send`, `recv`	extend / read from a labeled conversation with a model
`fork`, `clear`	isolate or reset a conversation
`endorse`	controlled, audited relabeling — declassification

2.6 The judgment form

Every evaluation step is stated as one judgment, threading four things through the program:

$$pc \vdash conv, e \Downarrow conv', v$$

- pc: the ambient label context this step runs under;
- $conv$: the conversation (message history plus its own label) coming in;
- e: the expression being evaluated;
- $conv', v$: the possibly-updated conversation and the resulting labeled value.

This is a *big-step* semantics: the judgment relates a whole expression to its final result in one step, defined by rules over its subexpressions.

2.7 The core rules look like ordinary lambda calculus

Similar to ordinary lambda calculus, an application evaluates the function, then the argument, then substitutes while carrying pc and $conv$ along for the ride.

Rule	What happens
`var`	look up a bound name; result label joins in the current pc
`lam`	a function value, labeled by the pc it was created under
`app`	evaluate e_1, then e_2 (threading $conv$), then apply

This is a simplified sketch for exposition, not the literal paper grammar — see `evaluator.ts` for the rule-by-rule implementation, each case commented with the exact paper section it implements.

2.8 $l : e$ — raising the label deliberately

This is how a program declares "what follows should be treated as at least this sensitive," independent of where the data physically came from:

```
{S} : (send secret_summary)
```

Evaluating e under $pc' = pc \sqcup l$ means every value e produces — and every effect it has — is checked as if it were at least as tainted as l. It is the explicit half of label propagation; the implicit half happens automatically every time a rule joins in the labels of whatever it read.

2.9 Runtime checks: `assert` and $e_1 \; ? \; e_2$

Labels are dynamic, so "is this flow allowed?" is a question the interpreter can only answer as it runs, not ahead of time.

- **`assert`** `e` — evaluate e; if the current label state violates the asserted condition, raise a `SecurityError` rather than proceed.
- $e_1 \; ? \; e_2$ — a labeled conditional: which branch fires (and under what raised pc) depends on a live $\sqsubseteq$ check, not a static type.

This is the difference between IFC done as a static type system (Volpano–Smith–Irvine style [4]) and IFC done dynamically, as here: no program is rejected before it runs, but a violation is caught the moment it would actually occur.

2.10 Every value always carries a label

Every value and not just top-level results carry a label. E.g., record fields, array elements, function closures each have their own label. This matters more than it might appear to. If reading a field out of a record simply returned the value as stored, without re-checking the current *pc*, a value could pass through a tainted context and come out looking untouched on the other side.

Getting this right for every place a value is *read back out* of something and not just where it is first produced turns out to be the single easiest place for an implementation to go quietly wrong; see Section 6.2.1.

3 Conversations and the Oracle

3.1 Conversations are first-class values

This is the calculus's real novelty over classical IFC: a conversation with a model is not a side channel outside the label system — it is a labeled value inside it. `LabeledConversation` pairs a message history with a single label: the join of every message ever sent into it. That label *is* the security boundary. Every rule that touches the conversation checks against it before acting, and updates it afterward to reflect whatever new taint just entered.

3.2 `send` — checked on the way out

```
send(e)   // checked: flowsTo(pc join label(v), conv.label)
```

Before a value can be appended to a conversation's history, the current *pc* joined with the value's own label must flow to the conversation's label. If it does not — if the program is trying to send something more sensitive than the conversation is cleared for — the send is rejected.

This is the rule that closes the paper's Fenton/Denning-style exfiltration gadget: a secret cannot smuggle itself out through what looks like an ordinary model call.

3.3 `recv` — the response inherits the conversation's taint

```
recv()   // parsed response evaluated at pc = conv.label
```

Whatever comes back from the model is parsed into code, and that code runs with pc set to the conversation's own label — *not* the caller's pc. A response that came out of an untrusted conversation is automatically treated as untrusted, no matter how innocuous it looks syntactically.

This closes the confinement failure from Section 1: generated code inherits exactly the trust level of where it came from.

3.4 `fork` — isolation by snapshot, not by gate

`send`, **`recv`**, and **`clear`** all check-then-update the conversation. **`fork`** does something different in kind: **`fork`** `{ body }` evaluates `body` against a *copy* of the current conversation. Whatever `body` does to that copy — including its own sends, receives, clears — is invisible to the caller, who gets back the original conversation, untouched. This is the entire mechanism behind sandboxing an untrusted sub-conversation: running it inside a fork *is* the sandbox.

3.5 `clear` — resetting without smuggling

One cannot simply empty a conversation's history and call it clean. **`clear`** resets `history` to `[]` *and* re-labels the conversation at the current pc. Dropping the messages but keeping the old, lower label would let a program launder a tainted conversation back down to a trust level it no longer deserves — clearing can only lower the bar as far as the current context actually justifies.

3.6 The oracle: where the one impure thing lives

An LLM call is nondeterministic. Everything else in the calculus is a pure function of its inputs. The calculus pushes that nondeterminism into a single explicit argument — the *oracle* — rather than letting it leak into the semantics implicitly. `evaluate` only ever calls `oracle.respond(history, callIndex)`, and only inside **`recv`**. Swap the oracle for a scripted, deterministic test double, and the entire interpreter becomes unit-testable without touching a real model.

3.7 Precisely stated confinement

Confinement is a property, not just an intuition: on the way out of **`recv`**, the produced value's label is always at least as high as the pc it was evaluated under.

$$pc \sqsubseteq \ell(V)$$

This is what makes "generated code cannot quietly act with more authority than the context it came from" a checkable statement rather than a hope. It is also exactly the

property that a naive environment-based implementation can accidentally violate — see Section 6.2.1.

3.8 Informal noninterference

The property all of this machinery is in service of: take two runs of the same program that differ only in their secret inputs. Noninterference says that, to an observer who can only see values at or below some trust level, the two runs are indistinguishable. Which means that nothing secret leaked into what that observer can tell, whether through data, control flow, or timing of visible effects.

The paper states and machine-checks three versions of this for the calculus — TIPNI, Insulated TIPNI, and oracular correctness — in Lean 4. What that does and does not mean for this TypeScript port is the subject of Section 7.

4 Declassification

4.1 Why you need a way out of strict noninterference

Strict noninterference is too strong for anything useful as sometimes crossing a label boundary is exactly the point. An agent reads a private document and is asked to produce a one-paragraph summary to share with a colleague. That summary is, by construction, derived from secret input — but it is also the deliberate output of the task. Refusing to ever let anything derived from a secret flow anywhere less secret makes the calculus useless for the actual job. A controlled, explicit exception is needed which we denote as *declassification*.

4.2 `endorse` — splitting the label in two

A blunt "just lower the label" is dangerous: it cannot distinguish relaxing *trust* from relaxing *secrecy*. `endorse` requires the lattice to be a *FactoredLattice*: one that decomposes a label along two independent axes, *integrity* (how much do we trust this value's origin?) and *confidentiality* (how sensitive is this value's content?). Endorsing can raise integrity without touching confidentiality, or the reverse — the two concerns need not move together.

4.3 `bounded_endorse` — a leakage budget, not a blank check

Any declassification is a controlled leak; the question is how much it can possibly leak. If a value is endorsed against a *fixed, static* trust domain of n possible options, decided at construction time rather than computed at runtime, the amount an adversary can learn from the outcome is bounded — at most $\log_2 n$ bits, the information content of "which of the n options was it." Compute the domain at runtime instead, and that bound no

longer holds. This is exactly why **`bounded_endorse`** is a *builder* that fixes its domain up front, rather than a value that could be handed a domain later.

4.4 `quarantine` — contain, then decide

Built from one primitive already introduced: **`fork`**. Running untrusted content inside a fork means anything it does — including any attempt to send data somewhere, or read something it should not — happens against a snapshot the caller never sees. **`quarantine`** is exactly this pattern, packaged: evaluate the untrusted expression inside its own forked conversation, and only what the caller explicitly extracts afterward, subject to its own label checks, can possibly escape.

4.5 `robust_endorse` — quarantine plus a controlled exit

Where **`quarantine`** alone isolates, **`robust_endorse`** combines that isolation with a deliberate, bounded relaxation of the result's label on the way out — the pattern for "let the model process this untrusted thing, and give me back a summary I can actually use," without ever letting the untrusted content itself touch anything outside the fork.

4.6 One shape, four rules

Stepping back, the conversation rules all follow the same pattern:

Rule	Check, then update
`send`	check outgoing flow — then join new taint into $conv.label$
`recv`	evaluate at $pc = conv.label$ — result inherits that taint by construction
`clear`	reset history — then re-label at the current pc
`fork`	no gate — snapshot instead, isolating rather than checking

`endorse` is the deliberate exception to "labels only ever go up": a named, audited place where a program can ask for less.

5 Related Work

LLMBDA is one of several provenance-based defences against prompt injection published within roughly eighteen months of one another, all responding to the same problem: an agent cannot tell untrusted data apart from a privileged instruction once both are just text in a context window. This section places the calculus against the four closest of them.

5.1 CaMeL: dual-LLM plus capability tracking

CaMeL [6] (Google, Google DeepMind, ETH Zürich) pairs a privileged planner model, which never reads untrusted data directly, with a quarantined model that reads it but

can only return values — the dual-LLM pattern LLMBDA's **`fork`**/**`quarantine`** can express as one policy among others rather than a fixed architecture. CAMEL has a public reference implementation and reports near-complete attack resistance with its policy checks on, but the original LLMBDA paper reproduces two concrete gaps in CAMEL's own tracker: a classic Fenton–Denning implicit-flow gadget (a secret-dependent branch that updates a variable only on the branch it does *not* take, which a purely dynamic monitor cannot taint), and an untracked Python retry loop that leaks one bit of a secret per iteration, launderable in full by an adaptive attacker. Turning CAMEL's policy checks on also costs roughly half its task-completion rate on the shared AgentDojo [8] banking suite (66.7% $\rightarrow$ 37.5% in the LLMBDA paper's reruns) — the utility/security trade-off LLMBDA's own benchmark is designed to close.

5.2 FIDES: dynamic taint tracking without control-flow secrecy

FIDES [7] (Microsoft Research) tracks confidentiality and integrity dynamically through a parametric planner loop, closer in spirit to LLMBDA's per-value labels than CAMEL's architecture is. Its confidentiality guarantee is, by the authors' own design, incomplete: it does not track secrets carried through data-dependent control flow, and it admits a bounded "small-domain values are safe" escape hatch. Both choices are defensible engineering trade-offs, but neither is auditable from a single run — there is no way to tell, after the fact, whether an admitted value was an intended downgrade or a leak wearing the same clothes. **`endorse`** is LLMBDA's answer to exactly this: a downgrade is only ever explicit, and Insulated TIPNI (Section 3.8) proves it cannot smuggle a change along an axis the call did not name.

5.3 LBAC/TypeGuard and tacit: static disciplines

Two contemporaneous systems take a different technical bet: static checking instead of dynamic tracking. LBAC/TypeGuard [9] (UC San Diego) type-checks agent-generated code against a Haskell `LIO`-based scaffolding before it runs; tacit [10] (EPFL) tracks capabilities through Scala 3's capture-checking type system. Neither has a noninterference-style proof for the composed, agent-generated application as a whole, and both show the same utility/security collapse pattern as CAMEL when enforcement is on (e.g. TypeGuard's reported 15/21 $\rightarrow$ 8/21 on a Slack-flavoured AgentDojo suite). They are not directly substitutable for LLMBDA's dynamic guarantee, though the original paper conjectures the two disciplines are complementary: static enforcement pre-execution, and a noninterference guarantee such as LLMBDA's for whatever dynamic behaviour a static check cannot rule out ahead of time.

5.4 Where LLMBDA differs

System	Core discipline	Soundness proof	Known/demonstrated gap
LLMBDA	Dynamic IFC, first-class conversation primitives	Machine-checked in Lean 4 (TIPNI, Insulated TIPNI, oracular correctness)	**`endorse`** escape hatch; external tool I/O not yet modelled
CAMEL [6]	Dual-LLM + capability-based control/data-flow extraction	None stated; authors call formalisation "a crucial direction" for future work	Untaken-branch implicit flow; untracked retry-loop bit leak
FIDES [7]	Dynamic taint tracking (confidentiality + integrity)	Formal model for the planner class; confidentiality guarantee deliberately incomplete	Ignores control-flow-carried secrets by design
LBAC/ TypeGuard [9]	Static typing of agent code against scaffolding	None (pre-execution type discipline, not a noninterference proof)	Catches pre-execution issues only; no residual dynamic-behaviour guarantee
tacit [10]	Static capability tracking via Scala 3 capture checking	None (capability discipline, not a confidentiality IFC proof)	Integrity/capability-flavoured; not a confidentiality guarantee

The comparison is not a claim that LLMBDA is unconditionally better — CAMEL and FIDES both have public reference implementations backed by production organisations today, while the LLMBDA paper's own Lean 4 artifact is, at the time of writing, available from the authors rather than publicly released (a gap this TypeScript port exists precisely to partially work around, without inheriting the proof).

The distinguishing LLMBDA claim is narrower and more specific: it is the only one of the five with an end-to-end machine-checked soundness proof for the whole composed system, including agent-generated code, and it reports the best published utility/security trade-off of the group on a shared benchmark.

6 The Port to TypeScript

6.1 Top-level architecture

A caller assembles two things and gets a pure function:

```
Model<L>                                    Oracle
(parse/serialise/primEval/toLabel)          (respond: history-> string)
                \                                 /
                 \                               /
     evaluate(model, oracle, run, pc, conv, expr, env)
                                 |
                                 v
                  EvalResult<L> = { conv, value }
```

where

- `Model<L>` supplies the policy — what labels look like, how to parse/serialise across the model boundary, what primitives exist.
- `Oracle` supplies the one impure thing. `evaluate` is otherwise a pure recursive function.

6.2 Architectural design decisions

The following subsections describe the major design decisions made in this port, and the rationale behind them. They are not a claim that the paper's original design was wrong, but rather a record of where the TypeScript implementation diverged from it, and why.

6.2.1 ADR-1: Environments implement substitution semantics

The paper's big-step semantics (§3/§5/§B) is defined over *substitution* ($e[x := e']$): a variable occurrence is *replaced* by the value it is bound to, so re-encountering that value under a subsequently-raised `pc` implicitly taints it via `⇓-Labelled`/`⇓-Lam`. This port uses environments (name → value maps) instead, which is the standard practical technique for implementing a substitution calculus — substitution itself is not something one wants to actually perform on every function application.

The catch: a naive environment lookup (`var`, record `.field` access) just returns whatever value was stored at bind time, unchanged. Under substitution semantics, a value read out from inside a `pc`-raised context is retroactively part of that context and picks up its taint; under naive environment lookup, it silently does not. This gap was found in `var`/`field` (`⇓-ArrayIndex` already did it correctly), fixed, and then found again independently in further spots across three rounds of a rule-by-rule audit — see the `evaluator.ts`/`lattice.ts` git history and `examples/var-pc-confinement.ts`.

Decision: Keep the environment-based implementation (rewriting to a substitution interpreter would be a much larger, higher-risk change for no semantic benefit), but treat "does this case need to re-join the ambient `pc` into the returned value's label" as a mandatory question for every case that *reads* a value out of an existing structure — environment, record, array — rather than constructing one fresh. This is now a standing check called out in `CLAUDE.md` for anyone adding a new evaluation rule.

6.2.2 ADR-2: Prelude functions are object-language closures

`fix`, `quarantine`, `robust_endorse`, and `bounded_endorse` (§C.5, §5.1, §E.2, §E.3) need to be callable from code the interpreter did not write — strings an LLM returns via `recv`, parsed by `Model.parse` into an `Expr`. Parsed code can only reference names bound in *that `Expr`'s environment*; it has no way to reach an arbitrary host-language (TS) function, no matter how it is registered.

`bounded_endorse` has an additional constraint: per §E.3, its trust domain must be a fixed, static list baked in at construction time — a runtime-computed domain would forfeit the paper's $\log_2 n$ leakage bound. That rules out implementing it as a single reusable `Expr` the way `fix`/`quarantine`/`robust_endorse` are.

Decision: Define `fix`, `quarantine`, and `robust_endorse` in `prelude.ts` as `Expr` values built from the same AST constructors used everywhere else (`ast.ts`), exposed as a name→`Expr` map via `Model.preludeSource`, and merged into scope for both the top-level program and

every parsed `recv` response (mirroring §3.3's `M.preludeEnv`). Define `bounded_endorse` as a *builder function* that takes the static trust domain at construction time and returns an `Expr`, rather than as a single fixed prelude entry.

6.2.3 ADR-3: No `weight`/probability tracking and no "fuel" argument

The paper's Lean 4 development carries a `weight`/probability parameter through its big-step judgment (needed for its probabilistic semantics and the machine-checked noninterference proofs) and a "fuel" argument (a standard technique to make an otherwise-partial evaluation function total for Lean's termination checker). Neither is part of what the calculus *computes* — both are proof-engineering artifacts of doing the semantics in a proof assistant.

Decision: Omit both from this port. `evaluate` is an ordinary (possibly non-terminating, like any interpreter) TypeScript function; it does not track a weight and does not take or decrement a fuel parameter.

6.2.4 ADR-4: Labels are homogeneous per run

`evaluate` is generic over the label type `L` (`Lattice<L>`/`Model<L>`), but composite runtime values — record fields, array elements — need to store labeled sub-values too. Threading `L` through `BareValue` generically (`Labeled<L>` everywhere a labeled value can appear, recursively) would work, but it means every data-shape type in `ast.ts` becomes generic in `L`, for a distinction that is never actually exercised: a single `evaluate()` call always closes over exactly one concrete `Model<L>`, so every label a given run ever produces already has the same concrete `L`.

Decision: `BareValue`'s record/array fields store `Labeled<unknown>` rather than `Labeled<L>`. Soundness of treating those `unknown` labels as the run's actual `L` comes from a single `asL<L>()` cast at the point in `evaluator.ts` where it matters, rather than from the type system proving it structurally throughout `ast.ts`.

6.2.5 ADR-5: `endorse`'s `FactoredLattice` requirement is a runtime check

`endorse` (§E.2/§E.3) needs to decompose a label into its integrity and confidentiality components, which only a `FactoredLattice<L, I, S>` (not every `Lattice<L>`) can do. Some `Model<L>` instances (e.g. ones built around a lattice that never needs `endorse`) only implement the plain `Lattice<L>` interface. The two options were: require every `Model<L>` to supply a `FactoredLattice`, even when unused, or let `Model<L>.lattice` be a plain `Lattice<L>` and treat the extra factoring capability as something checked when `endorse` is actually reached.

Decision: `Model<L>.lattice` is typed as `Lattice<L>`. `endorse` requires it to also implement `FactoredLattice`; `evaluator.ts`'s `asFactoredLattice` does a runtime capability check (testing for `toIntegrity`/ `toConfidentiality`/`pair`) and throws `RuntimeError` if it is missing, rather than the type system statically forbidding `endorse` on a non-factored model.

6.2.6 ADR-6: Default `parse`/`serialise` is naive JSON

Every `send`/`recv` crosses the `Expr`↔string boundary through `Model.parse`/`Model.serialise`. The paper's §7.3 flags that a production deployment needs a grammar-constrained parser — one that can only ever produce a well-formed `Expr`, closing off a class of oracle-side attacks where a

malicious or malfunctioning LLM response parses into something unintended. Building that parser is real, nontrivial work orthogonal to what this port exists to demonstrate (the evaluation semantics).

Decision: Ship `defaultParse`/`defaultSerialise` as `JSON.parse`/`JSON.stringify` wrapped in a try/catch, and document — in `README.md`'s design-choices list, `CLAUDE.md`, and here — that this is a known, deliberate gap (Article 4), not a claim that this is production-ready parsing.

6.2.7 ADR-7: `fork` snapshots the conversation instead of gating access to it

`send`, `recv`, and `clear` all gate access to `LabeledConversation<L>` by checking `flowsTo(pc, conv.label)` before touching it, then updating `conv.label` to reflect the new taint. `fork` is different in kind: its job (§5.1, backing `quarantine` in `prelude.ts`) is to run a sub-program against a conversation the caller is walled off from, not to check a flow condition on shared state.

Decision: `fork` does not gate the conversation at all. `expr.body` evaluates against a *copy* of `conv`; anything that body does — including its own `send`/`recv`/`clear` calls — is invisible to the caller, who gets back the original, pre-fork `conv` unchanged. `quarantine` is implemented entirely as "run untrusted code inside a `fork`" — there is no separate sandboxing mechanism.

6.3 Worked example: postcode extraction

The minimal end-to-end wiring, from `examples/postcode.ts`:

```
const model = useModel();               // labels, parse/serialise,
   primEval
const oracle = scriptedOracle([...]);   // deterministic canned
   responses

const { value } = runProgram(
  model, oracle, bottomPc, emptyConv, program
);
```

The program asks the model to extract a postcode from user-provided text, receives a response through **`recv`**, and the result carries the join of every label the text passed through — no separate tracking step, just the semantics running.

6.4 Worked example: the Fenton–Denning leak test

A regression test demonstrates that **`send`**'s check actually rejects the attack it is meant to. The example constructs the classic covert-channel gadget — branch on a secret, take an action visible on an untrusted channel depending on which branch ran — and asserts that the interpreter throws a `SecurityError` rather than letting the send through.

Because it is regression-tested against the pre-fix source too, it is not just "this looks right" — it is confirmed to actually catch the bug it names.

6.5 Testing discipline

Three rules, enforced in continuous integration:

- **Regression test proven to fail first.** A fix is not done until there is a test that fails against the pre-fix code and passes against the fix.
- **100% statement/function/line coverage, 99% branch.** Every line in `src/` is exercised by an example; the build fails if that regresses.
- **A real typecheck gate.** A separate `tsconfig` typechecks `examples//test/` too — a deliberately-injected type error there was found to slip past the plain build silently.

This discipline is why the audit history reads "eight bugs found across three passes," not "looked correct on first read."

7 Limitations and Future Work

7.1 What this port does not prove

The paper's central results — TIPNI, Insulated TIPNI, oracular correctness — are machine-checked in the paper's own **Lean 4** development. Porting the algorithm to TypeScript does not port the proof.

What this repository offers instead is the same evaluation rules, regression-tested against the paper's own worked examples, including its named leak gadgets. **"The interpreter rejects the leak"** is evidence. **"The interpreter is noninterferent"** is a claim only the Lean development is entitled to make.

7.2 Future work

Here is some future work to bulletproof the implementation:

A conformance-vector harness. Diff this interpreter's output against the Lean-side `peval` on shared $(program, oracle) \to (conversation, value)$ triples. Given how many bugs hand-auditing already found, likely the highest-value item on this list.

Property-based testing. Use `fast-check` to generate pairs of programs related by the paper's $\sim_m$ equivalence, and check their traces stay compatible under a scripted oracle — real evidence, not a proof, but a good way to hunt for divergences intuition alone will not find.

A randori-style agent harness. Practice against a mock world state, then regenerate and run for real — evidence the port is usable for something, not just faithful to the semantics on paper.

A real oracle. Only `scriptedOracle`/`ruleOracle` — test doubles — exist today. A production oracle backed by an actual LLM API is a thin adapter away, and would let the whole apparatus run against real model output.

7.3 Further reading

On the LLMBDA calculus

- Garby, Gordon & Sands, *The* LLMBDA *Calculus*, arXiv:2602.20064.
- This repository's `README.md`, `ARCHITECTURE.md`, and `CONSTITUTION.md`.
- `docs/adr/` — the individual implementation decisions and the reasoning behind each.

Classical information-flow-control background

- Denning, *A Lattice Model of Secure Information Flow*, Communications of the ACM, 1976.
- Fenton, *Memoryless Subsystems*, The Computer Journal, 1974.
- Volpano, Smith & Irvine, *A Sound Type System for Secure Flow Analysis*, Journal of Computer Security, 1996.
- Sabelfeld & Myers, *Language-Based Information-Flow Security*, IEEE Journal on Selected Areas in Communications, 2003.

7.4 Appendix A: file structure

The code is structured as follows:

File	Role
`ast.ts`	Expr/Value AST and builder functions
`lattice.ts`	`Lattice<L>/FactoredLattice` plus concrete instances
`model.ts`	parse/serialise/primEval/toLabel — the policy boundary
`oracle.ts`	the pluggable, injectable source of nondeterminism
`evaluator.ts`	the big-step semantics, rule by rule
`prelude.ts`	`fix/quarantine/endorse` helpers as real object-language code
`errors.ts`	`SecurityError` vs `RuntimeError`

References

[1] Garby, Z., Gordon, A., & Sands, D. *The LLMbda Calculus: AI Agents, Conversations, and Information Flow*. arXiv:2602.20064, 2026.

[2] Denning, D. E. *A Lattice Model of Secure Information Flow*. Communications of the ACM, 19(5), 236–243, 1976.

[3] Fenton, J. S. *Memoryless Subsystems*. The Computer Journal, 17(2), 143–147, 1974.

[4] Volpano, D., Smith, G., & Irvine, C. *A Sound Type System for Secure Flow Analysis*. Journal of Computer Security, 4(2–3), 167–187, 1996.

[5] Sabelfeld, A., & Myers, A. C. *Language-Based Information-Flow Security*. IEEE Journal on Selected Areas in Communications, 21(1), 5–19, 2003.

[6] Debenedetti, E., Shumailov, I., Fan, T., Hayes, J., Carlini, N., Fabian, D., Kern, C., Shi, C., Terzis, A., & Tramèr, F. *Defeating Prompt Injections by Design*. arXiv:2503.18813, 2025.

[7] Costa, M., Köpf, B., Kolluri, A., Paverd, A., Russinovich, M., Salem, A., Tople, S., Wutschitz, L., & Zanella-Béguelin, S. *Securing AI Agents with Information-Flow Control*. arXiv:2505.23643, 2025.

[8] Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., & Tramèr, F. *AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents*. arXiv:2406.13352, 2024.

[9] Zhou, T., D'Antoni, L., & Polikarpova, N. *Language-Based Agent Control*. arXiv:2605.12863, 2026.

[10] Odersky, M., Zhao, Y., Xu, Y., Bračevac, O., & Pham, C. N. *Tracking Capabilities for Safer Agents*. arXiv:2603.00991, 2026.